

SQL and PL/SQL — Complete Question Bank Answers

Q1 Define Relational Database Query.

Definition:

A **relational database query** is a request made to a relational database management system (RDBMS) to retrieve, insert, update, or delete data stored in one or more relations (tables). SQL (Structured Query Language) is the standard language used to write such queries.

Key Points:

- A query is written using SQL and is processed by the RDBMS.
- It uses a combination of **relational algebra** and **relational calculus** concepts.
- The basic form of a retrieval query is: `SELECT ... FROM ... WHERE ...`
- A query operates on one or more tables and returns a result as a new relation (table).
- SQL is case-insensitive — *SELECT*, *select*, and *Select* all work the same way.
- Queries can be simple (select all rows) or complex (involving joins, subqueries, aggregates).

Example:

```
-- Find names of all employees
SELECT ename
FROM employee;

-- Find employees with salary > 20000
SELECT ename, salary
FROM employee
WHERE salary > 20000;
```

Q2 SQL Queries on Employee(Empno, Ename, DeptNo, Salary) & Department(DeptNo, Dname)

1) List employee names of Computer Department:

```
SELECT e.Ename
FROM Employee e, Department d
WHERE e.DeptNo = d.DeptNo
      AND d.Dname = 'Computer';
```

2) Find average salary of each department:

```
SELECT d.Dname, AVG(e.salary) AS Avg_Salary
FROM Employee e, Department d
WHERE e.DeptNo = d.DeptNo
GROUP BY d.Dname;
```

3) Find department name of employee 'Amit':

```
SELECT d.Dname
FROM Employee e, Department d
WHERE e.DeptNo = d.DeptNo
      AND e.Ename = 'Amit';
```

■ Note: JOIN is done by matching DeptNo in both tables. AVG() is an aggregate function that computes average. GROUP BY groups rows with same DeptNo together.

Q3 Examine the role of Integrity Constraints in DBMS — Types with Examples

What are Integrity Constraints?

Integrity constraints are rules enforced on data columns of a table to ensure the **accuracy, consistency, and reliability** of data in the database. They limit the type of data that can go into a table. They can be column-level or table-level.

Types of Integrity Constraints:

1. Domain Integrity Constraints

- **NOT NULL:** Ensures a column cannot have NULL value. Example: Roll_No INT NOT NULL
- **UNIQUE:** Ensures all values in a column are different. Example: Emp_ID VARCHAR(20) UNIQUE
- **DEFAULT:** Provides a default value when no value is specified. Example: Marks INT DEFAULT 0
- **CHECK:** Ensures values satisfy a condition. Example: Age INT CHECK(Age >= 18)

2. Entity Integrity Constraints

- **PRIMARY KEY:** Uniquely identifies each row; cannot be NULL.
- Each table can have only one primary key.
- Example: PRIMARY KEY (Emp_ID)

3. Referential Integrity Constraints

- **FOREIGN KEY:** Links two tables — the foreign key in one table refers to the primary key in another.
- **CASCADE DELETE:** If a parent row is deleted, child rows are automatically deleted.
- **CASCADE UPDATE:** If parent key changes, child rows update automatically.

Example — Create Table with Constraints:

```
CREATE TABLE instructor (  
    ID          CHAR(5),  
    name        VARCHAR(20) NOT NULL,  
    dept_name    VARCHAR(20),  
    salary       NUMERIC(8,2) CHECK(salary > 0),  
    PRIMARY KEY (ID),  
    FOREIGN KEY (dept_name) REFERENCES department  
);
```

Q4 Apply Aggregate Functions in SQL — Usage with Examples

What are Aggregate Functions?

Aggregate functions operate on a set of values (a column) and return a single summarized value. They are used with SELECT, GROUP BY, and HAVING clauses.

List of Aggregate Functions:

Function	Purpose	Example
COUNT()	Count number of rows	COUNT(*) — total rows
SUM()	Sum of values	SUM(salary)
AVG()	Average of values	AVG(salary)

MAX()	Largest value	MAX(salary)
MIN()	Smallest value	MIN(salary)

Examples with Queries:

```
-- 1. Count total employees
SELECT COUNT(*) FROM employee;

-- 2. Find average salary
SELECT AVG(salary) FROM employee;

-- 3. Find max and min salary
SELECT MAX(salary), MIN(salary) FROM employee;

-- 4. Total salary per department (GROUP BY)
SELECT dept_name, SUM(salary) AS Total_Salary
FROM employee
GROUP BY dept_name;

-- 5. Departments with avg salary > 50000 (HAVING)
SELECT dept_name, AVG(salary) AS Avg_Salary
FROM employee
GROUP BY dept_name
HAVING AVG(salary) > 50000;
```

■ Note: WHERE filters rows BEFORE grouping. HAVING filters groups AFTER grouping. Aggregate functions ignore NULL values (except COUNT(*)).

Q5

Demonstrate the Rename Operation in Relational Algebra with Examples

What is Rename Operation?

The **rename operation** allows us to give a new name to a relation or its attributes. In relational algebra, it is denoted by ρ (**rho**). In SQL, the **AS** keyword is used to rename relations (tables) or attributes (columns).

Why Use Rename?

- To simplify long table/column names in queries.
- To perform self-joins (joining a table with itself).
- To give meaningful names to computed columns.
- To use the same table multiple times in one query.

Syntax in SQL (AS Clause):

```
-- Rename column
SELECT salary/12 AS monthly_salary
FROM employee;

-- Rename table (alias)
SELECT e.ename, d.dname
FROM employee AS e, department AS d
WHERE e.deptno = d.deptno;

-- Self-join using rename (find employees with higher salary
-- than some employee in 'IT' dept)
SELECT DISTINCT T.ename
```

```
FROM employee AS T, employee AS S
WHERE T.salary > S.salary
AND S.deptname = 'IT';
```

■ Note: In Oracle SQL, the keyword AS must be omitted for table aliases (e.g., FROM employee e). For column aliases, AS is optional in most databases.

Q6 Classify Cursors in PL/SQL. State Lifecycle of Cursor with Proper Syntax.

What is a Cursor?

A cursor is a **temporary work area** (pointer) created in system memory when a SQL statement is executed. It holds the result set of a SELECT query and allows row-by-row processing. A cursor can hold multiple rows but processes only **one row at a time**.

Types of Cursors:

1. Implicit Cursor

- Automatically created by the DBMS when DML statements (INSERT, UPDATE, DELETE, SELECT) are executed.
- The programmer does not explicitly declare it.
- Attributes: %FOUND, %NOTFOUND, %ROWCOUNT, %ISOPEN
- %FOUND / %NOTFOUND return TRUE/FALSE. %ROWCOUNT returns number of rows affected.

2. Explicit Cursor

- Defined by the programmer to store the result of a SELECT query.
- Used when fine-grained (row-by-row) control is needed.
- Must be declared, opened, fetched, and closed manually.

Lifecycle of Explicit Cursor (4 Steps):

Step	Action	Syntax
1. DECLARE	Define the cursor with a SELECT statement	CURSOR c1 IS SELECT ...;
2. OPEN	Open cursor and execute SELECT query	OPEN c1;
3. FETCH	Retrieve one row at a time into variables	FETCH c1 INTO var1, var2;
4. CLOSE	Release memory / close cursor	CLOSE c1;

Example — Explicit Cursor to display employees:

```
DROP PROCEDURE IF EXISTS pcursor;
CREATE PROCEDURE pcursor()
BEGIN
    DECLARE fn VARCHAR(30);
    DECLARE ln VARCHAR(30);
    DECLARE cur1 CURSOR FOR
        SELECT first_name, last_name
        FROM emp_tbl
        WHERE salary > 1000;
    OPEN cur1;
```

```

read_loop: LOOP
    FETCH cur1 INTO fn, ln;
    SELECT CONCAT(fn, ' ', ln) AS name;
END LOOP;
CLOSE cur1;
END;

```

Q7 Demonstrate INNER JOIN and OUTER JOIN in SQL with Syntax and Examples

What is a JOIN?

A JOIN combines rows from two or more tables based on a related column. It returns a result relation formed by combining the tuples.

1. INNER JOIN

- Returns only the rows that have **matching values** in both tables.
- Non-matching rows are excluded from the result.

```

-- Syntax
SELECT columns
FROM table1
INNER JOIN table2
ON table1.column = table2.column;

-- Example: Get employee names and their department names
SELECT e.ename, d.dname
FROM employee e
INNER JOIN department d
ON e.deptno = d.deptno;

```

2. OUTER JOIN (Three Types):

a) LEFT OUTER JOIN

- Returns all rows from the **left table**, and matching rows from the right table.
- If no match, NULL is placed for right table columns.

```

SELECT e.ename, d.dname
FROM employee e
LEFT OUTER JOIN department d
ON e.deptno = d.deptno;

```

b) RIGHT OUTER JOIN

- Returns all rows from the **right table**, matching rows from left; NULLs for unmatched left rows.

```

SELECT e.ename, d.dname
FROM employee e
RIGHT OUTER JOIN department d
ON e.deptno = d.deptno;

```

c) FULL OUTER JOIN

- Returns all rows from **both tables**; NULLs where no match exists on either side.

```

SELECT e.ename, d.dname
FROM employee e
FULL OUTER JOIN department d
ON e.deptno = d.deptno;

```

What is a Trigger?

A trigger is a **special stored procedure** that automatically runs (fires) when a specific event (INSERT, UPDATE, or DELETE) occurs on a table. The user does not call a trigger directly — the database fires it automatically.

Purpose / Uses of Triggers:

- Enforce business rules automatically (e.g., salary increase limit).
- Maintain audit trails — log changes to important tables.
- Enforce referential integrity and data consistency.
- Generate derived column values automatically.
- Prevent invalid transactions (e.g., negative balance).
- Synchronize related tables automatically.

Types of Triggers:

- **BEFORE Trigger:** Fires before the DML operation on the table.
- **AFTER Trigger:** Fires after the DML operation completes.
- **ROW-LEVEL Trigger:** Fires once for each affected row.
- **STATEMENT-LEVEL Trigger:** Fires once per DML statement regardless of rows.
- MySQL supports: BEFORE INSERT, AFTER INSERT, BEFORE UPDATE, AFTER UPDATE, BEFORE DELETE, AFTER DELETE.

Syntax:

```
CREATE TRIGGER trigger_name
{BEFORE | AFTER}
{INSERT | UPDATE | DELETE}
ON table_name
FOR EACH ROW
BEGIN
    -- trigger body (SQL statements)
END;
```

Example — Trigger as Recycle Bin:

When a row is deleted from EMP, copy it to EMP_BIN:

```
CREATE TRIGGER emp_recycle_bin
BEFORE DELETE ON emp
FOR EACH ROW
BEGIN
    INSERT INTO emp_bin (eno, ename, salary)
    VALUES (OLD.eno, OLD.ename, OLD.salary);
END;
```

■ Note: OLD refers to the row before change. NEW refers to the row after change. In INSERT trigger, only NEW is used. In DELETE trigger, only OLD is used. In UPDATE trigger, both OLD and NEW are used.

i) Display location of department where employee 'Ram' works:

```
SELECT a.Dept_location
FROM Emp e, Dept d, Address a
WHERE e.Dept_no = d.Dept_no
      AND d.Dept_name = a.Dept_name
      AND e.Emp_name = 'Ram';
```

ii) Create a view to store total employees per department in ascending order:

```
CREATE VIEW dept_emp_count AS
  SELECT d.Dept_name, COUNT(e.Emp_no) AS Total_Employees
  FROM Dept d
  LEFT OUTER JOIN Emp e
  ON d.Dept_no = e.Dept_no
  GROUP BY d.Dept_name
  ORDER BY Total_Employees ASC;
```

iii) Find department name where no employee is working:

```
SELECT Dept_name
FROM Dept
WHERE Dept_no NOT IN (
  SELECT DISTINCT Dept_no
  FROM Emp
);
```

iv) Find salaries department-wise where salary > 5000000:

```
SELECT d.Dept_name, e.Emp_name, e.salary
FROM Emp e, Dept d
WHERE e.Dept_no = d.Dept_no
      AND e.salary > 5000000
ORDER BY d.Dept_name;
```

Q10

Differentiate between Tuple Relational Calculus and Domain Relational Calculus with Examples

Introduction:

Relational calculus is a **non-procedural** query language that tells WHAT data to retrieve without specifying HOW. It has two variants: Tuple Relational Calculus (TRC) and Domain Relational Calculus (DRC).

Point	Tuple Relational Calculus (TRC)	Domain Relational Calculus (DRC)
Definition	Variables represent tuples (entire rows)	Variables represent domain values (individual attributes)
Variable Type	Tuple variable t ranges over entire rows	Domain variables for each attribute separately
Notation	{ t P(t) }	{ P(x1,...xn) }
Readability	Easier to read, closer to SQL	More verbose, low-level
Expressiveness	Less granular control on attributes	More precise control on each attribute
TRC Example	{ t t ∈ Emp ∧ t.sal > 20000 }	Not applicable
DRC Example	Not applicable	{ ∈ Emp ∧ s > 20000 }
Based on	SQL is inspired by TRC	QBE (Query By Example) uses DRC

Example Comparison:

Find names of employees with salary > 20000:

```
-- TRC: { t | t ∈ Employee ∧ t.salary > 20000 }
-- (t is a tuple variable representing a whole row)

-- DRC: { <name> | ∃ empno, deptno, salary
--             (<empno, name, deptno, salary> ∈ Employee
--             ∧ salary > 20000) }
-- (each variable represents one domain/attribute value)
```

Q11 Explain the Importance of a Primary Key in a Relational Database

Definition:

A **Primary Key** is a column (or combination of columns) that uniquely identifies each row in a table. It enforces the **Entity Integrity Constraint** — ensuring no two rows have the same primary key value and no primary key can be NULL.

Importance of Primary Key:

- **Unique Identification:** Each row can be uniquely identified — no two rows have the same PK value.
- **No NULL Values:** Primary key cannot be NULL — ensures every row is identifiable.
- **Enforces Entity Integrity:** Prevents duplicate or incomplete records from being inserted.
- **Basis for Relationships:** Primary keys are referenced by Foreign Keys to link tables together.
- **Index Creation:** DBMS automatically creates a unique index on the primary key for faster lookups.
- **Query Optimization:** Queries using PK in WHERE clause are faster because of indexing.

Example:

```
CREATE TABLE employee (
    empno    INT          NOT NULL,
    ename     VARCHAR(30) NOT NULL,
    salary    NUMERIC(8,2),
    PRIMARY KEY (empno)
);

-- empno uniquely identifies every employee.
-- NULL or duplicate empno values are not allowed.
```

Q12 Examine Set Operations in SQL — UNION and INTERSECT with Examples

Introduction:

SQL supports set operations that work on two relations and return a new relation. The three main set operations are UNION, INTERSECT, and EXCEPT. By default, each removes duplicates. To keep duplicates, use UNION ALL, INTERSECT ALL, EXCEPT ALL.

1. UNION

- Returns all distinct rows from **both** relations combined.
- Eliminates duplicate rows automatically.
- Both tables must have the same number and type of columns.

```
-- Find all customers who have a loan OR an account (or both)
(SELECT customer_name FROM depositor)
UNION
(SELECT customer_name FROM borrower);
```



```
-- To keep duplicates:
(SELECT customer_name FROM depositor)
UNION ALL
(SELECT customer_name FROM borrower);
```

2. INTERSECT

- Returns only rows that appear in **both** relations.
- Finds common records between two result sets.

```
-- Find customers who have BOTH a loan AND an account
(SELECT customer_name FROM depositor)
INTERSECT
(SELECT customer_name FROM borrower);
```

3. EXCEPT (MINUS in Oracle)

- Returns rows in the first relation that are **not** in the second relation.

```
-- Find customers who have an account but NO loan
(SELECT customer_name FROM depositor)
EXCEPT
(SELECT customer_name FROM borrower);
```

■ Note: INTERSECT and EXCEPT are not supported in all versions of MySQL. They can be simulated using IN / NOT IN subqueries.

Q13

SQL Queries on EmployeeDetails(EmpID, FullName, MangrID, DoJ) & EmployeeSalary(EmpID, Project, Salary)

Q1. Count employees working in project 'P1':

```
SELECT COUNT(*) AS Emp_Count
FROM EmployeeSalary
WHERE Project = 'P1';
```

Q2. Fetch employee names with salary between 5000 and 10000:

```
SELECT ed.FullName
FROM EmployeeDetails ed, EmployeeSalary es
WHERE ed.EmpID = es.EmpID
      AND es.Salary >= 5000
      AND es.Salary <= 10000;
```

Q3. Fetch employee names and salary — include employees even if salary record is missing (LEFT JOIN):

```
SELECT ed.FullName, es.Salary
FROM EmployeeDetails ed
LEFT OUTER JOIN EmployeeSalary es
ON ed.EmpID = es.EmpID;
```

Q4. Fetch common records between two tables:

```
SELECT ed.EmpID, ed.FullName, es.Project, es.Salary
FROM EmployeeDetails ed
INNER JOIN EmployeeSalary es
ON ed.EmpID = es.EmpID;
```

Q14

Apply PL/SQL to Develop Database Programs Integrating SQL with Procedural Constructs

What is PL/SQL?

PL/SQL (Procedural Language/SQL) is an extension of SQL that adds **procedural constructs** like variables, loops, conditions, exception handling, and subprograms to SQL. It allows writing database programs that combine SQL queries with programming logic.

Structure of a PL/SQL Block:

```
-- Basic PL/SQL Block Structure:
DECLARE
    -- variable declarations
BEGIN
    -- executable statements (SQL + procedural code)
EXCEPTION
    -- error handling
END;
```

Example 1 — Simple Procedure to Add Two Numbers:

```
DELIMITER //
CREATE PROCEDURE addp()
BEGIN
    DECLARE a, b, c INT;
    SET a = 10;
    SET b = 20;
    SET c = a + b;
    SELECT CONCAT('Sum is: ', c);
END//
DELIMITER ;
CALL addp();
```

Example 2 — Procedure with IN/OUT Parameters:

```
DELIMITER //
CREATE PROCEDURE get_name(IN id INT, OUT ename VARCHAR(30))
BEGIN
    SELECT emp_name INTO ename
    FROM employee
    WHERE empno = id;
END//
DELIMITER ;
CALL get_name(101, @n);
SELECT @n;
```

Example 3 — Conditional Logic (Find Maximum of Three Numbers):

```
CREATE PROCEDURE maxthree()
BEGIN
    DECLARE a, b, c INT;
    SET a = 100; SET b = 50; SET c = 40;
    IF (a > b AND a > c) THEN
        SELECT CONCAT('Maximum is: ', a);
    ELSEIF (b > a AND b > c) THEN
        SELECT CONCAT('Maximum is: ', b);
    ELSE
        SELECT CONCAT('Maximum is: ', c);
    END IF;
END;
```

Q15 Examine Characteristics of PL/SQL and How They Enhance Database Programming

Characteristics of PL/SQL:

- **Block Structure:** PL/SQL programs are organized into blocks (DECLARE, BEGIN, EXCEPTION, END). This makes code modular and readable.
- **Procedural Constructs:** Supports IF-THEN-ELSE, loops (LOOP, WHILE, FOR), variables, and constants — unlike plain SQL.
- **Exception Handling:** Built-in error handling using EXCEPTION section to catch and manage runtime errors gracefully.
- **SQL Integration:** SQL statements (SELECT, INSERT, UPDATE, DELETE) can be used directly inside PL/SQL blocks.
- **Stored Procedures and Functions:** Reusable code units compiled and stored in the database — reduces network traffic.
- **Cursors:** Provides row-by-row processing of query results using explicit and implicit cursors.
- **Triggers:** Allows automatic execution of code in response to database events (INSERT, UPDATE, DELETE).
- **Performance:** PL/SQL code is sent as a single block to the server, reducing round-trips and improving efficiency.

How PL/SQL Enhances Efficiency:

- Reduces network overhead by sending entire blocks to the database server at once.
- Stored procedures are precompiled and cached — faster execution on subsequent calls.
- Business logic stored in the database is consistent across all applications.
- Exception handling prevents application crashes due to data errors.

Q16 Differentiate between SQL and PL/SQL — Features, Execution Model, and Use Cases

Point	SQL	PL/SQL
Full Form	Structured Query Language	Procedural Language / SQL
Nature	Declarative — specifies WHAT to do	Procedural — specifies HOW to do it
Execution	One statement at a time	Entire block sent and executed together
Variables	No variables	Supports variables, constants, data types
Control Flow	No IF/ELSE, loops	Supports IF-ELSE, LOOP, WHILE, FOR
Error Handling	No built-in exception handling	EXCEPTION block for error handling
Use Case	Simple queries (SELECT, INSERT, UPDATE)	Complex logic: procedures, functions, triggers
Reusability	Cannot store reusable code in DB	Stored procedures/functions reusable

■ Summary: SQL is used for data manipulation and retrieval. PL/SQL extends SQL with programming capabilities to build complex applications stored and run inside the database.

Q17 Demonstrate the Use of %TYPE and %ROWTYPE Data Types in PL/SQL

1. %TYPE Attribute

%TYPE is used to declare a variable with the **same data type** as a specific column in a table. This ensures the variable type automatically matches the table column type.

- Avoids type mismatch errors.
- If the column data type changes, the variable type changes automatically.
- Syntax: variable_name table_name.column_name%TYPE;

```
-- %TYPE Example
DECLARE
    v_ename employee.ename%TYPE; -- same type as ename column
    v_salary employee.salary%TYPE; -- same type as salary column
BEGIN
    SELECT ename, salary
    INTO v_ename, v_salary
    FROM employee
    WHERE empno = 101;
    SELECT CONCAT('Name: ', v_ename, ', Salary: ', v_salary);
END;
```

2. %ROWTYPE Attribute

%ROWTYPE declares a variable (record) that has the **same structure as an entire row** of a table — it contains all the columns of that table as fields.

- Useful when fetching and storing an entire row from a table.
- Reduces the need to declare individual variables for each column.
- Syntax: record_name table_name%ROWTYPE;

```
-- %ROWTYPE Example
DECLARE
    emp_rec employee%ROWTYPE; -- holds one full row of employee table
BEGIN
    SELECT *
    INTO emp_rec
    FROM employee
    WHERE empno = 101;
    -- Access individual fields using dot notation
    SELECT CONCAT('Name: ', emp_rec.ename,
                ', Dept: ', emp_rec.deptno);
END;
```

Q18 Classify Cursor Types in PL/SQL and Elaborate Functioning of Explicit Cursor

Cursor Classification:

Type	Description	When Used
Implicit Cursor	Auto-created by DBMS for DML statements	INSERT, UPDATE, DELETE, single-row SELECT
Explicit Cursor	Defined by programmer for multi-row SELECT	When row-by-row processing is needed

Functioning of Explicit Cursor (Detailed):

- **Step 1 — DECLARE:** Associate cursor with a SELECT statement. Memory is NOT yet allocated.
- **Step 2 — OPEN:** Execute the SELECT query; result set is stored in cursor memory.

- **Step 3 — FETCH:** Read one row at a time from the cursor into local variables.
- **Step 4 — CLOSE:** Release the cursor memory and resources.
- **Attributes:** %ISOPEN (cursor open?), %FOUND (row fetched?), %NOTFOUND (no more rows?), %ROWCOUNT (rows fetched so far).

Complete Explicit Cursor Example:

```
CREATE PROCEDURE display_emp_cursor()
BEGIN
    DECLARE v_name  VARCHAR(30);
    DECLARE v_sal   NUMERIC(8,2);
    DECLARE done    INT DEFAULT 0;
    -- Step 1: DECLARE cursor
    DECLARE emp_cur CURSOR FOR
        SELECT ename, salary
        FROM employee
        WHERE salary > 20000;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
    -- Step 2: OPEN cursor
    OPEN emp_cur;
    read_loop: LOOP
        -- Step 3: FETCH one row
        FETCH emp_cur INTO v_name, v_sal;
        IF done = 1 THEN
            LEAVE read_loop;
        END IF;
        SELECT CONCAT(v_name, ' earns ', v_sal);
    END LOOP;
    -- Step 4: CLOSE cursor
    CLOSE emp_cur;
END;
```

Q19 Illustrate COMMIT, ROLLBACK, and SAVEPOINT with SQL/PL/SQL Examples

What are TCL Commands?

TCL (Transaction Control Language) commands are used to manage transactions — groups of SQL statements that execute as a single unit. They ensure **data consistency** and allow controlled saving or reversal of changes.

1. COMMIT

- Permanently saves all changes made in the current transaction to the database.
- Once committed, changes cannot be rolled back.
- All DML changes (INSERT/UPDATE/DELETE) are temporary until COMMIT is issued.

```
INSERT INTO employee VALUES (201, 'Ravi', 10, 35000);
UPDATE employee SET salary = 40000 WHERE empno = 201;
COMMIT; -- Changes are now permanently saved
```

2. ROLLBACK

- Undoes all changes made in the current transaction — restores to last committed state.
- Used when an error occurs and we want to cancel incomplete work.

```
DELETE FROM employee WHERE empno = 201;
-- Oops! Wrong delete
ROLLBACK; -- Delete is undone; row is back
```

```
-- Rollback to a specific savepoint:  
ROLLBACK TO sp1;
```

3. SAVEPOINT

- Creates a named checkpoint within a transaction.
- Allows partial rollback — roll back to the savepoint instead of the entire transaction.

```
INSERT INTO employee VALUES (202, 'Priya', 10, 30000);  
SAVEPOINT sp1;    -- Checkpoint created here  
  
INSERT INTO employee VALUES (203, 'Raj', 20, 25000);  
-- Problem with this insert  
ROLLBACK TO sp1;  -- Only second insert is undone  
COMMIT;           -- First insert (202) is saved
```

Q20 Design a PL/SQL Script to Display the Series: 99, 96, 93, ... 9, 6, 3

Analysis of the Series:

- The series starts at 99 and ends at 3.
- Each number decreases by 3 (common difference = -3).
- All numbers are multiples of 3.
- We need a loop that starts at 99 and decrements by 3 until it reaches 3.

PL/SQL Script using WHILE loop:

```
DELIMITER //  
CREATE PROCEDURE display_series()  
BEGIN  
    DECLARE n INT DEFAULT 99;  
    WHILE n >= 3 DO  
        SELECT n AS series_value;  
        SET n = n - 3;  
    END WHILE;  
END//  
DELIMITER ;  
CALL display_series();
```

Alternative using a single SELECT with CONCAT (display as one line):

```
DELIMITER //  
CREATE PROCEDURE series_concat()  
BEGIN  
    DECLARE n INT DEFAULT 99;  
    DECLARE res VARCHAR(500) DEFAULT '';  
    WHILE n >= 3 DO  
        SET res = CONCAT(res, n, IF(n > 3, ', ', ''));  
        SET n = n - 3;  
    END WHILE;  
    SELECT res AS 'Series Output';  
END//  
DELIMITER ;  
CALL series_concat();
```

■ Output: 99, 96, 93, 90, 87, 84, 81, 78, 75, 72, 69, 66, 63, 60, 57, 54, 51, 48, 45, 42, 39, 36, 33, 30, 27, 24, 21, 18, 15, 12, 9, 6, 3

Q21 List Different SQL Data Types with Examples

SQL Data Types are categorized into three groups:

Data Type	Description	Example
CHAR(n)	Fixed-length character string of length n	CHAR(5) → 'Hello'
VARCHAR(n)	Variable-length character string, max n chars	VARCHAR(50) → 'Pune'
INT / INTEGER	Whole numbers (4 bytes)	age INT → 25
SMALLINT	Smaller integer (2 bytes)	marks SMALLINT → 95
NUMERIC(p,d)	Fixed-point number with p digits, d after decimal	salary NUMERIC(8,2) → 35000.50
FLOAT(p)	Floating-point number with precision p	price FLOAT → 99.99
DATE	Date value (YYYY-MM-DD)	dob DATE → 2001-05-15
BOOLEAN	TRUE or FALSE	active BOOLEAN → TRUE
BLOB	Binary Large Object — for images/files	photo BLOB
TEXT	Large text data	description TEXT

Q22 Define Stored Procedures and Functions in PL/SQL — Purpose in Database Applications

Stored Procedure:

A stored procedure is a **named, precompiled block of PL/SQL code** stored in the database that performs a specific action. It is invoked using the CALL statement and does **not return a value directly**.

- Can have IN, OUT, or INOUT parameters.
- Used to perform actions like INSERT, UPDATE, DELETE, or business logic.
- Called by: CALL procedure_name(arguments);

```
CREATE PROCEDURE add_employee(IN pname VARCHAR(30), IN psal NUMERIC(8,2))
BEGIN
    INSERT INTO employee(ename, salary) VALUES(pname, psal);
END;
CALL add_employee('Amit', 35000);
```

Stored Function:

A stored function is a **named PL/SQL block** that always **returns a single value**. It can be used directly inside SQL statements like ordinary SQL functions.

- Uses the RETURN keyword to return a value.
- Parameters work like IN parameters of a procedure.
- Called inside SQL: SELECT function_name(args) FROM ...;

```
CREATE FUNCTION get_annual_salary(emp_id INT)
RETURNS NUMERIC(10,2)
BEGIN
    DECLARE ann_sal NUMERIC(10,2);
    SELECT salary * 12 INTO ann_sal
```

```

    FROM employee WHERE empno = emp_id;
    RETURN ann_sal;
END;

-- Use in SELECT:
SELECT get_annual_salary(101);

```

Q23 Examine Different Types of JOINS in SQL with Usage and Examples

Overview of JOINS:

A JOIN combines rows from two or more tables based on a related column. Different JOIN types control how unmatched rows from either table are handled.

JOIN Type	Description	Use Case
INNER JOIN	Returns only matching rows from both tables	Most common; excludes unmatched rows
LEFT OUTER JOIN	All rows from left table + matching from right	Useful to find unmatched rows in left
RIGHT OUTER JOIN	All rows from right table + matching from left	Useful to find unmatched rows in right
FULL OUTER JOIN	All rows from both tables; NULLs for unmatched	Complete picture of both tables
NATURAL JOIN	Joins on all matching column names automatically	No ON clause needed; risky if column names differ
CROSS JOIN	Cartesian product — every row of table1 × table2	Returns n*m rows; rarely used alone
SELF JOIN	A table joined with itself using aliases	Used for hierarchical or comparison queries

Examples:

```

-- INNER JOIN
SELECT e.ename, d.dname
FROM employee e INNER JOIN department d ON e.deptno = d.deptno;

-- LEFT OUTER JOIN (shows employees even without a department)
SELECT e.ename, d.dname
FROM employee e LEFT OUTER JOIN department d ON e.deptno = d.deptno;

-- SELF JOIN (find employees whose salary > their manager's)
SELECT e.ename AS Employee, m.ename AS Manager
FROM employee e JOIN employee m ON e.mgrno = m.empno
WHERE e.salary > m.salary;

-- CROSS JOIN
SELECT * FROM employee CROSS JOIN department;

```

Q24 Analyze SQL Set Operations and Set Membership Operators with Examples

Set Operations in SQL:

SQL set operations work on two result sets (relations) and combine them. Both SELECT statements must have the same number and compatible column types.

- **UNION:** Combines all distinct rows from both queries.
- **UNION ALL:** Combines all rows including duplicates.
- **INTERSECT:** Returns rows common to both queries.
- **EXCEPT (MINUS):** Returns rows in first query but not in second.

Set Membership Operators:

- **IN:** Tests if a value is in a set (result of a subquery). Equivalent to = SOME.
- **NOT IN:** Tests if a value is NOT in a set. Equivalent to != ALL.
- **EXISTS:** Returns TRUE if subquery returns at least one row.
- **NOT EXISTS:** Returns TRUE if subquery returns no rows.
- **SOME / ANY:** Compares value to each row of a subquery — true if condition holds for at least one.
- **ALL:** Compares value to each row of a subquery — true only if condition holds for ALL rows.

Examples:

```
-- IN: Find employees working in 'CS' or 'IT' departments
SELECT ename FROM employee
WHERE deptno IN (SELECT deptno FROM department
                WHERE dname IN ('CS', 'IT'));

-- NOT IN: Find departments with no employees
SELECT dname FROM department
WHERE deptno NOT IN (SELECT DISTINCT deptno FROM employee);

-- EXISTS: Find employees who have a manager
SELECT ename FROM employee e
WHERE EXISTS (SELECT 1 FROM employee m WHERE m.empno = e.mgrno);

-- > ALL: Find employee earning more than every IT dept employee
SELECT ename FROM employee
WHERE salary > ALL (SELECT salary FROM employee
                   WHERE deptno = 10);
```

Q25 Write SQL Queries using Aggregate Functions and GROUP BY to Summarize Data

Table: Employee(empno, ename, deptno, salary)

```
-- 1. Count total number of employees
SELECT COUNT(*) AS Total_Employees
FROM employee;

-- 2. Find total salary paid by the company
SELECT SUM(salary) AS Total_Salary
FROM employee;

-- 3. Find department-wise average salary
SELECT deptno, AVG(salary) AS Avg_Salary
FROM employee
GROUP BY deptno;

-- 4. Find max and min salary in each department
```

```

SELECT deptno,
       MAX(salary) AS Max_Salary,
       MIN(salary) AS Min_Salary
FROM employee
GROUP BY deptno;

-- 5. Count employees in each department
SELECT deptno, COUNT(*) AS Emp_Count
FROM employee
GROUP BY deptno;

-- 6. Departments where average salary > 30000 (HAVING)
SELECT deptno, AVG(salary) AS Avg_Salary
FROM employee
GROUP BY deptno
HAVING AVG(salary) > 30000;

-- 7. Find salary summary for each dept (with dept name)
SELECT d.dname,
       COUNT(e.empno) AS Emp_Count,
       SUM(e.salary) AS Total_Salary,
       AVG(e.salary) AS Avg_Salary
FROM employee e, department d
WHERE e.deptno = d.deptno
GROUP BY d.dname;

```

Q26 Construct SQL Queries using Nested Subqueries to Retrieve Complex Data

What is a Nested Subquery?

A **subquery** is a SELECT statement nested inside another SQL statement. It is evaluated first and its result is used by the outer query. Subqueries can be used in WHERE, FROM, or HAVING clauses.

Types of Subqueries:

- **Single-row subquery:** Returns one row — used with =, >, < etc.
- **Multi-row subquery:** Returns multiple rows — used with IN, ANY, ALL.
- **Correlated subquery:** References column from outer query — re-evaluated for each outer row.

Examples:

```

-- 1. Find employees who earn more than the average salary
SELECT ename, salary
FROM employee
WHERE salary > (SELECT AVG(salary) FROM employee);

-- 2. Find employees in 'Computer' department
SELECT ename
FROM employee
WHERE deptno = (
    SELECT deptno FROM department WHERE dname = 'Computer'
);

-- 3. Find department with highest average salary
SELECT dname
FROM department d, employee e

```

```

WHERE d.deptno = e.deptno
GROUP BY d.dname
HAVING AVG(e.salary) = (
    SELECT MAX(avg_sal)
    FROM (SELECT AVG(salary) AS avg_sal FROM employee GROUP BY deptno) AS t
);

-- 4. Find employees who work in the same dept as 'Amit'
SELECT ename
FROM employee
WHERE deptno = (SELECT deptno FROM employee WHERE ename = 'Amit');

```

Q27 Design and Create a View and an Index for a Database Application

PART A — VIEW

What is a View?

A **view** is a virtual table derived from one or more base tables using a SELECT query. It does not store data physically — only the view definition is stored. It improves data abstraction, security, and simplicity.

- Hides sensitive columns from unauthorized users.
- Simplifies complex multi-table queries into a simple virtual table.
- Changes in base tables are reflected in the view automatically.

```

-- Create a view to show department-wise employee summary
CREATE VIEW dept_summary AS
    SELECT d.dname AS Department,
           COUNT(e.empno) AS Total_Employees,
           AVG(e.salary) AS Avg_Salary,
           SUM(e.salary) AS Total_Salary
    FROM department d
    LEFT JOIN employee e ON d.deptno = e.deptno
    GROUP BY d.dname;

-- Use the view
SELECT * FROM dept_summary
WHERE Total_Employees > 5;

-- Drop a view
DROP VIEW dept_summary;

```

PART B — INDEX

What is an Index?

An **index** is a data structure that speeds up data retrieval in a table. It works like a book index — instead of scanning the entire table, the DBMS uses the index to find records quickly. However, INSERTs and UPDATEs become slightly slower.

```

-- Create an index on employee table for faster search by ename
CREATE INDEX idx_ename ON employee(ename);

-- Create a unique index on Emp_ID
CREATE UNIQUE INDEX idx_empid ON employee(empno);

-- Drop an index

```

```
ALTER TABLE employee DROP INDEX idx_ename;
```

Q28**Write SQL Statements to Create and Use Sequences for Automatic Unique Value Generation****What is a Sequence?**

A **sequence** is a database object that generates unique, sequential numeric values automatically. It is mainly used to auto-generate primary key values, removing the need to manually specify IDs.

AUTO_INCREMENT in MySQL (equivalent to Sequence):

```
-- Create table with AUTO_INCREMENT (MySQL's sequence equivalent)
CREATE TABLE employee (
    empno    INT NOT NULL AUTO_INCREMENT,
    ename    VARCHAR(30) NOT NULL,
    salary   NUMERIC(8,2),
    PRIMARY KEY (empno)
);

-- Insert without specifying empno - it is auto-generated
INSERT INTO employee (ename, salary) VALUES ('Amit', 35000);
INSERT INTO employee (ename, salary) VALUES ('Priya', 42000);
INSERT INTO employee (ename, salary) VALUES ('Raj', 28000);

-- View result
SELECT * FROM employee;
-- Output: empno = 1, 2, 3 generated automatically
```

Oracle-style Sequence (for reference):

```
-- Create a sequence (Oracle syntax)
CREATE SEQUENCE emp_seq
    START WITH 1
    INCREMENT BY 1
    MAXVALUE 9999
    NOCYCLE;

-- Use the sequence
INSERT INTO employee VALUES (emp_seq.NEXTVAL, 'Amit', 35000);
```

Q29**Compare Stored Procedures and Functions in PL/SQL**

Point	Stored Procedure	Stored Function
Return Value	Does not return a value directly; uses OUT parameter	Always returns exactly one value using RETURN keyword
Call Method	Called using: CALL procedure_name(args)	Called inside SQL: SELECT function_name(args)
Purpose	Perform an action (INSERT, UPDATE, DELETE)	Compute and return a value
Parameters	Supports IN, OUT, INOUT parameters	Only IN parameters (input only)

Point	Stored Procedure	Stored Function
Use in SQL	Cannot be used directly inside SELECT statement	Can be used inside SELECT, WHERE, HAVING clauses
DML Inside	Can contain DML statements (INSERT, UPDATE, DELETE)	Generally should not modify data (read-only recommended)
Syntax	CREATE PROCEDURE ... BEGIN ... END	CREATE FUNCTION ... RETURNS type BEGIN RETURN val; END
Example	CALL add_employee('Amit', 35000);	SELECT get_annual_salary(101);

Q30 List the Different SQL Commands and Classify Them Based on Functionality

SQL Commands are classified into 4 categories:

Category	Full Form	Commands	Purpose
DDL	Data Definition Language	CREATE, ALTER, DROP, RENAME, TRUNCATE	Define or modify database structure (schema)
DML	Data Manipulation Language	SELECT, INSERT, UPDATE, DELETE	Retrieve or modify data in tables
DCL	Data Control Language	GRANT, REVOKE	Control access permissions for users
TCL	Transaction Control Language	COMMIT, ROLLBACK, SAVEPOINT	Manage transactions and data consistency

Quick Examples of Each Category:

```
-- DDL: Create a table
CREATE TABLE employee (empno INT, ename VARCHAR(30));

-- DDL: Modify structure
ALTER TABLE employee ADD salary NUMERIC(8,2);

-- DML: Insert data
INSERT INTO employee VALUES (101, 'Amit', 35000);

-- DML: Retrieve data
SELECT * FROM employee WHERE salary > 30000;

-- DCL: Grant access
GRANT SELECT ON employee TO user1;

-- TCL: Save transaction
COMMIT;
```